Institución Universitaria	GUÍA DE TRABAJO Tecnología en Desarrollo de Software	Código	FDE 048
		Versión	03
		Fecha	2009-06-09

1. IDENTIFICACIÓN

Asignatura	*Programación de Software*							Guía No.	2
Área	Básicas de la Tecnología					Nivel		4	
Código	580304006					Pensum		1	
Correquisito(s)	*580506004*					Prerrequisito(s)		NA	
Créditos	4	TPS	64	TIS	128	TPT		TIT	
TRABAJO INDEPENDIENTE						TRABAJO PRESENCIAL			
Trabajo Teórico	X	Trabajo Práctico		X		Trabajo Teórico		Trabajo Práctico	X

2. IDENTIFICACIÓN

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
Analiza, diseña e implementa aplicaciones de desarrollo de software en capas, utilizando la programación orientada a objetos con énfasis en bases de datos a partir de los requerimientos del cliente, para la solución de problemas en los sistemas de información.	✓ Preferir software para la realización de aplicaciones donde se resuelvan problema o requerimientos solicitados por el cliente de manera modular. ✓ Uso de SGBD/DBMS[1] e integración a una aplicación que requiere almacenar información de manera recurrente. ✓ Uso de la arquitectura ADO.NET para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) directamente, gestionando la base de datos de forma tradicional en una aplicación WEB.	✓ Organiza la capa de datos para la interaccionan de los formularios de la aplicación con la información existente en los SGBD, utilizando el lenguaje SQL para la manipulación de esta. ✓ Plantear diferentes medios de abstracción e interacción entre la información existente en SGBD y el usuario.

3. RECURSOS REQUERIDOS

Sala de sistemas con 40 equipos con Visual Studio 2026 y SQL Server, 1 proyector de video y marcadores varios.
Material bibliográfico

4. PROCEDIMIENTO

Programación a 3 capas
ADO.NET - (Web MVC y CRUD)

4.1 Consultar

1. ¿Qué es una programación a tres capas? Beneficios y Desventajas.
2. ¿En qué consiste la capa de base de datos?
3. ¿Qué es el patrón de arquitectura MVC en programación de software? ¿Qué hace cada una de ellas?
4. ¿Qué es CRUD en programación? ¿Qué hace cada una de ellas?
5. ¿Qué es la tecnología **ADO.NET?**
6. ¿Cuál es la arquitectura de ADO.NET?
7. ¿Cuál es el proceso para abrir y cerrar una conexión a una Base de datos SQL 2005/2008 o superior desde C#?
8. ¿Cuáles son los objetos básicos para manipular la información en una base de datos de ADO.NET?
9. ¿Cuáles son las librerías de .NET e instrucciones de C# necesarias para realizar una conexión a bases de datos SQL 2005/2008 o superior?
10. ¿Como documentar Propiedades y Métodos en una clase en un proyecto .NET
11. ¿Cuál es la función de los objetos: **¿SqlDataReader, SqlDataAdapter, SqlDataSet y DataTable** en la tecnología ADO.NET?
12. ¿En la tecnología .NET? Cuáles son las propiedades y los métodos necesarios para utilizar un: **SqlConnection, SqlCommand, sqlDataReader y DataSet?**
13. ¿Cuales son los tres tipos de ejecución de un **SqlCommand? , cual es la función de cada uno.**
14. ¿Cuáles son las similitudes y las diferencias entre un **SqlDataReader** y un **SqlDataSet?**
15. ¿En la tecnología .NET, Cómo se llena un: **sqlDataReader** y ***DataSet?.***
16. ¿Cómo crear un usuario en Microsoft SQL Server e instanciarlo a una Base de datos determinada?

4.2 Problema para solucionar

SuperMercado SAS requiere de un formulario para registrar a los Usuarios. Debe contener el código de usuario, el número y el tipo de documento, el/los nombre, el/los apellidos, el género (10, Masculino o 20, Femenino), activo y observaciones. El formulario debe contener la forma de poder realizar las opciones de buscar, ingresar, modificar, grabar y cancelar. Los tipos de documentos con sus respectivos id: (1, Cédula Ciudadanía), (3, Cédula Extranjería), (6, Pasaporte), (7, NIT), (8, NUIP), (9, Permiso de Protección Temporal).

[1] Sistema de Gestión de Bases de Datos (en inglés: DataBase Management System)

Powered by CamScanner

Actividad # 1. Bases de Datos:

1.1. Crear una carpeta con el nombre: ***Practica2_SuperMkdo***

1.2. Crear una subcarpeta con nombre: ***BasedeDatos***

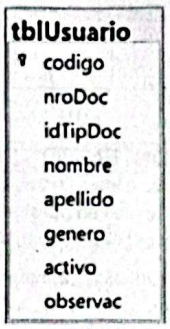
1.3. Crear un script de base de datos en el DBMS SQL Server y guárdalo en la subcarpeta creada en el numeral anterior, con nombre: ***bdSuperMkdo***; dicho script debe contener los siguientes ítems:

i. Crear y usar la base de datos: **bdSuperMkdo** (colocar la instrucción: **go** al final de cada estructura, instrucción o grupo de instrucciones)

```
Create Database bdSuperMkdo
go
USE bdSuperMkdo
go
```

ii. Crear la tabla: **tblUsuario**

```
Create Table tblUsuario
(  codigo int Primary Key NOT NULL,
   nroDoc int NOT NULL,
   idTipDoc int NOT NULL,
   nombre varchar(50) NOT NULL,
   apellido varchar(50) NOT NULL,
   genero int NOT NULL,
   activo bit default 1 NOT NULL,
   observac varchar(500) NULL
)
go
```



iii. Ingresar datos a la tabla:

```
--============ tblUsuario ============
INSERT INTO tblUsuario VALUES
 (30001, 1020001000, 7, 'Claudia',    'Aristizábal Rúa',   20, 1, 'Ppto Particip'),
 (30002, 1020001001, 8, 'Mario A.',   'Martínez Rios',     10, 1, 'Fondo EPM'),
 (30003, 1020001002, 1, 'Natalia',    'Castrillón Caguán', 20, 1, null),
 (30004, 1020001003, 1, 'Rubén D.',   'Soto Rios',         10, 1, 'Pruebas de todo'),
 (30005, 1020001004, 3, 'Scott',      'Rangún Suteu',      10, 1, 'Reserva'),
 (30006, 1020001005, 9, 'Jazmín A.',  'Arango Marín',      20, 1, 'Ppto Particip.'),
 (30007, 1020001006, 6, 'Cristina M.','Soto Diaz',         20, 1,'Suspende Matric')
go
```

iv. Crear Consultas almacenadas (Stored Procedures) :

```
Create Procedure USP_Usu_BuscarXCodigo
@intCodigo int
AS
 BEGIN
  SELECT nroDoc, idTipDoc, nombre, apellido, genero, Activo, isnull( observac,'' ) observac
    FROM tblUsuario
   WHERE codigo = @intCodigo
  -- EXEC USP_Usu_BuscarXCodigo 30001;
 END
go
```

```
Create Procedure USP_Usu_Grabar
@intNroDoc   int,
@intTipDoc   int,
@strNombre   VARCHAR(50),
@strApelli   VARCHAR(50),
@intGenero   int,
@bitActivo   BIT,
@strObserv   VARCHAR(500)
As
 BEGIN
  IF EXISTS( SELECT * FROM tblUsuario
             WHERE nroDoc = @intNroDoc )
   BEGIN
    SELECT -1 AS Rpta
    Return
   END
  ELSE
   BEGIN
    BEGIN TRANSACTION tx
     Declare @intNewCod int
     Set @intNewCod = ( select max( codigo ) + 1
                        from tblUsuario )
    INSERT INTO tblUsuario VALUES ( @intNewCod,
          @intNroDoc, @intTipDoc, @strNombre, @strApelli,
```

```
Create Procedure USP_Usu_Modificar
@intCodigo   int,
@intNroDoc   int,
@intTipDoc   int,
@strNombre   VARCHAR(50),
@strApelli   VARCHAR(50),
@intGenero   int,
@bitActivo   BIT,
@strObserv   VARCHAR(500)
As
 BEGIN
  IF EXISTS( SELECT * FROM tblUsuario
             WHERE nroDoc = @intNroDoc and codigo <> @intCodigo )
   BEGIN
    SELECT -1 AS Rpta
    Return
   END
  ELSE
   BEGIN
    BEGIN TRANSACTION tx
     UPDATE tblUsuario SET
          nroDoc = @intNroDoc, idTipDoc = @intTipDoc,
          nombre = @strNombre, apellido = @strApelli,
          genero = @intGenero, Activo = @bitActivo,
          Observac = @strObserv
```


Powered by CamScanner

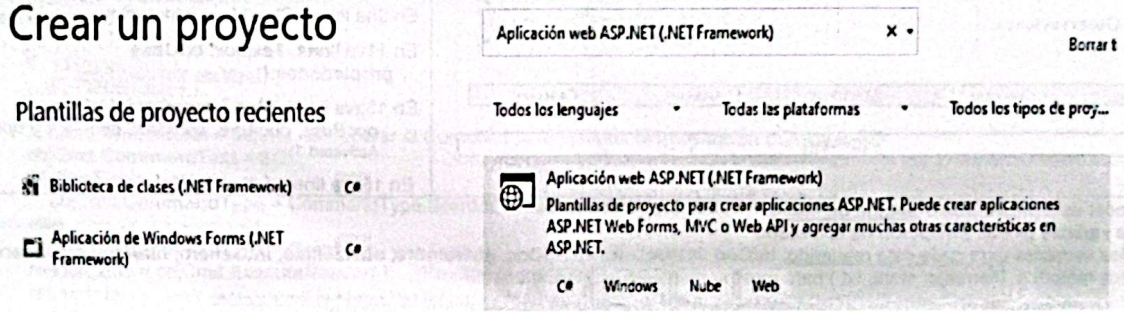
```
        @intGenero, @bitActivo, @strObserv );
    IF ( @@ERROR > 0 )
     BEGIN
      ROLLBACK TRANSACTION tx
      SELECT 0 AS Rpta
      Return
    END
    COMMIT TRANSACTION tx
    SELECT @intNewCod AS Rpta
    Return
  END
  /* EXEC USP_Usu_Grabar '70500600', 1, 'Andrés', 'Duque', 10, 1, null;
    EXEC USP_Usu_BuscarXcodigo '30008';      */
 END
go
```

```
    WHERE codigo = @intCodigo
    IF ( @@ERROR > 0 )
     Begin
      ROLLBACK TRANSACTION tx
      SELECT 0 AS Rpta
      Return
    End
    COMMIT TRANSACTION tx
    SELECT @intCodigo  AS Rpta
    Return
  END
  /* EXEC USP_Usu_BuscarXcodigo '30008';
 EXEC USP_Usu_Modificar '30008', '70500600', 6, 'Antonia', 'Duque', 20, 1, 'test';
   EXEC USP_Usu_BuscarXcodigo '30008';     */
 END
go
```

Actividad 2. Crear Proyecto WEB en C# con formulario

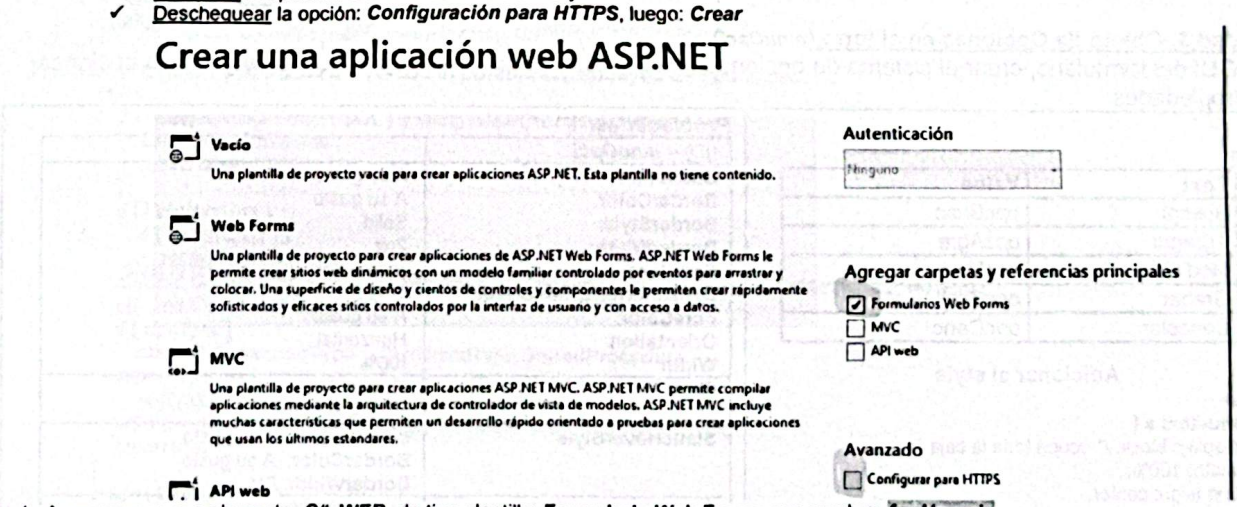
2.1. Aplicación para entorno WEB para realizar la captura y muestra de datos, utilizando las librerías de clases implementadas anteriormente

a. En Microsoft Visual Studio 2026, utilizar la opción: Crear un Proyecto, buscando o utilizando la plantilla de proyecto: *Aplicación web ASP.NET (.NET Framework)* en C#, con nombre: ***webSuperMkdo***, en la subcarpeta creada en la actividad 1(1.1). *Crear.*

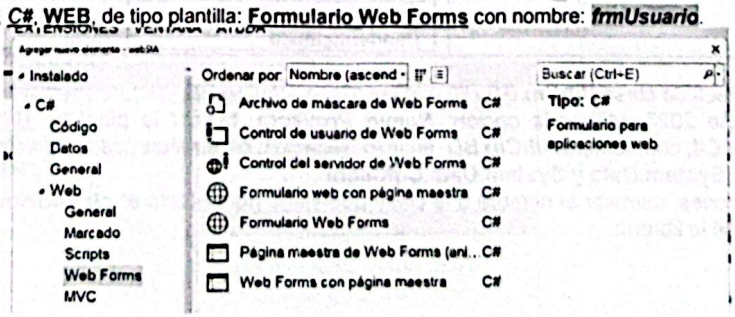


Te lleva a la ventana de: *Crear una aplicación web ASP.NET*, en ella seleccionar: ***Empty*** (Vacío) y

- ✓ Chequear la opción: ***Formularios Web Forms*** y
- ✓ Deschequear la opción: ***Configuración para HTTPS***, luego: ***Crear***

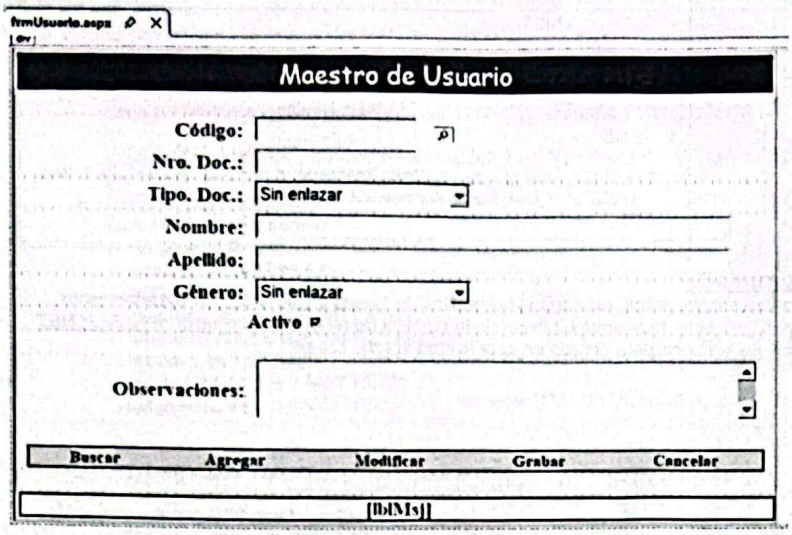


b. Agregar un nuevo elemento, C#, WEB, de tipo plantilla: **Formulario Web Forms** con nombre: ***frmUsuario***.



Powered by CamScanner

c. En el proyecto: Agregar -> Nueva carpeta, con nombre: ***Imagenes***; en ella adicionar la imagen entregada por el docente. (Pararse en la carpeta creada, Agregar -> Elemento existente, buscar en donde tiene la imagen y adicionarla).
d. En ***frmUsuario*** diseñar los objetos de entrada y salida propuestos en la siguiente imagen, utilizando una tabla de 15*2, con ancho del 90%, centrada.



Form: ***frmUsuario***
(*Titulo/title*): Maestro de Usuario

En 1ra línea: Unirla en un td, (Text): ***Maestro de Usuario***

Ubicar los siguientes objetos en 2do td de cada línea

En 3ra línea, *Textbox:* **txtCodI**, ReadOnly: true, luego un, *ImageButton:* **ibtnBusc**, en propiedad: *ImageUrl* seleccionar la imagen de la carpeta: Imagenes.

En 4ta línea, *Textbox*: **txtNDoc**

En 5ta línea, *DropDownList*: **ddlTDoc**

En 6ta línea, *Textbox*: **txtNomb**

En 7ma línea, *Textbox*: **txtApel**

En 8va línea, *DropDownList*: **ddlGene**

En 9na línea, *CheckBox*: **chkActi** Text: Activo y TextAlign: Left)

En 11va línea, *Textbox*: **txtObse**
propiedades:(TextMode: MultiLine y Rows: 3)

En 13ava línea, Menú: **mnuOpci**, *Values*:
opcBusc, opcAgre, opcModi, opcGrab y opcCanc. (Ver Actividad 3)

En 15ava línea, Label: **lblMsj, (center, red)**

e. Establecer esta página como página de inicio
f. Crear la variable global: ***intOpcion*** de tipo **static**
g. Crear las variables para cada dato requerido: **intCod, intNroDoc, intTipDoc, strNombre, strApellido, intGenero, blnAct y strObservac.**
h. Crear los métodos: **Mensaje(** string txt **)** para mostrar mensajes en: *lblMsj*
i. Crear los métodos: **llenarComboGenero() y llenarComboTipoDoc()** de forma manual utilizando la instrucción:
this.ddlGene.Items.Add(new ListItem("Seleccionar...", "0")); con los datos en el enunciado respectivamente.
j. En el método Page_Load, en la negación del postBack, invocar los métodos: **llenarComboProg( ) y llenarComboTipoDoc().**

Actividad 3. Objeto de Opciones en el form (*mnuOpci*)

En la GUI del formulario, crear el sistema de opciones con el objeto: Menú, ***mnuOpci;*** con las siguientes opciones y propiedades:

Items

Opciones	Text	Value
	Buscar	opcBusc
	Agregar	opcAgre
	Modificar	opcModi
	Grabar	opcGrab
	Cancelar	opcCanc

Adicionar el style

```
<style>
  .menu-item a {
      display: block; /* ocupe toda la caja */
      width: 100%;
      text-align: center;
  }
</style>
```

Propiedades:

(ID) = *mnuOpci*	
BackColor:	A su gusto
BorderColor:	A su gusto
BorderStyle:	Solid
BorderWidth:	2px
CssClass:	***menu-item*** //Creado en el style
DynamicHorizontalOffset:	2
ForeColor:	A su gusto
Orientation:	Horizontal
Width:	100%
Font:	**Bold** = True **Size** = Medium
StaticHoverStyle	**BackColor:** A su gusto **BorderColor:** A su gusto **BorderWidth:** 2px
StaticMenuItemStyle	**HorizontalPadding:** 20px
RenderingMode	Table

Actividad 4. Crear la biblioteca de clase (***libCnxBD.dll***) con las Clase: ***clsCnxBD.***
a En Microsoft Visual Studio 2026, utilizar la opción: ***Nuevo Proyecto***, buscar la plantilla: ***Biblioteca de clases (.NET Framework)***, con ***Visual C#***, con nombre: ***libCnxBD***, en la localización: carpeta creada en la actividad 1(1.1). Aceptar
b. Usar (***using***) las librerías: **System.Data y System.Data.SqlClient**
c. En el explorador de soluciones, cambiar el nombre a la clase que viene por defecto a: ***clsConexion***, grabar para que quede así guardado el archivo de la librería.

Powered by CamScanner

d. Crear constructor donde inicialice ***blnHayCnx*** en falso, y crear los objetos: ***objCnx*** desde SqlConnection, ***objCmd*** desde SqlCommand y ***objParam*** desde SqlParameter.

e. Crear el método privado: ***abrirCnx***

```
if ( string.IsNullOrEmpty( cadCnx ) )
   return false;
objCnx.ConnectionString = cadCnx;
try
{  objCnx.Open( );
   blnHayCnx = true;
   return true;
}
catch ( Exception ex )
{  Error = "Error, reintentar";
   blnHayCnx = false;
   return false;
}
```

f. Crear el método público: ***cerrarCnx***

```
try
{  objCnx.Close( );
   objCnx = null;
   blnHayCnx = false;
}
catch
{  Error = "Problemas en liberación de la conexión." }
```

clsCnxBD		
-	blnHayCnx	: bool
-	objCnx	: SqlConnection
-	objCmd	: SqlCommand
-	objParam	: SqlParameter
+	SQL	: string ↓
+	cadCnx	: string ↓
+	readerLleno	: SqlDataReader ↑
+	vrRpta	: object ↑
+	Error	: string ↑
-	abrirCnx( )	: bool
+	Consultar(bool tipoEjec)	: bool
+	vrUnico(bool tipoEjec)	: bool
+	execCmd(bool tipoEjec)	: bool
+	addParam(string nombreParam, object vrParam)	: bool
+	clsCnxBD( ) // constructor	

g. Crear el método público: ***Consultar*** con el parámetro, ***tipoEjec*** de tipo lógico

```
try
{  if ( string.IsNullOrEmpty( SQL ) || string.IsNullOrEmpty( cadCnx ) )
   {  Error = "Faltan datos";
      return false;
   }
   if ( ! blnHayCnx )
      if ( ! abrirCnx( ) )
         return false;
   objCmd.Connection = objCnx;  //Preparar el Comando para ejecutar la transacción SQL en la BD
   objCmd.CommandText = SQL;
   if ( tipoEjec)
      objCmd.CommandType = CommandType.StoredProcedure;  //Asincrónico
   else
      objCmd.CommandType = CommandType.Text;              //Sincronico
   readerLleno = objCmd.ExecuteReader( );  //Realizar la transacción en la BD
   return true;
}
catch ( Exception ex )
{  Error = ex.Message;
   return false;
}
```

h. Crear el método público: ***vrUnico*** con el parámetro, ***tipoEjec*** de tipo lógico

```
try
{  if ( string.IsNullOrEmpty( SQL ) || string.IsNullOrEmpty( cadCnx ) )
   {  Error = "Faltan datos";
      return false;
   }
   if ( ! blnHayCnx )
      if ( ! abrirCnx( ) )
         return false;
   objCmd.Connection = objCnx;  //Preparar el Comando para ejecutar la transacción SQL en la BD
   objCmd.CommandText = SQL;
   if ( tipoEjec )
      objCmd.CommandType = CommandType.StoredProcedure;
   else
      objCmd.CommandType = CommandType.Text;
   vrRpta = objCmd.ExecuteScalar();  //Realizar la transacción en la BD
   return true;
}
catch ( Exception ex )
{  Error = ex.Message;
   return false;
}
```

i. Crear el método público: ***execCmd*** con el parámetro, ***tipoEjec*** de tipo lógico

```
try
{  if ( string.IsNullOrEmpty( SQL ) || string.IsNullOrEmpty( cadCnx ) )
   {  Error = "Faltan datos";
      return false;
   }
   if ( ! blnHayCnx )
      if ( ! abrirCnx( ) )
         return false;
```


Powered by CamScanner

```
    objCmd.Connection = objCnx;    //Preparar el Comando para ejecutar la transacción SQL en la BD
    objCmd.CommandText = SQL;
    if ( tipoEjec )
       objCmd.CommandType = CommandType.StoredProcedure;
    else
       objCmd.CommandType = CommandType.Text;
    objCmd.ExecuteNonQuery( );  //Realizar la transacción en la BD
    return true;
  }
  catch ( Exception ex )
  {  Error = ex.Message;
     return false;
  }  string nombreParam, object
```

j. Crear el método público: ***addParam*** con los parámetros, ***nombreParam*** de tipo cadena y ***vrParam*** de tipo object

```
try {
   objCmd.Parameters.AddWithValue( nombreParam, vrParam ); //Adicionar el parámetro al command
   objParam = new SqlParameter( );    //Crear un nuevo parámetro para el siguiente parámetro
   return true;
}
catch
{  Error = "Error en ingreso de dato.";
   return false;
}
```

Actividad 5. En el Proyecto *web ASP.NET*: ***webSuperMkdo***, crear la clase local: ***clsUsuario***

a En el explorador de soluciones, en el proyecto, agregar una *nueva carpeta* con nombre: ***Clases***. En dicha carpeta Agregar una nueva clase local con nombre: ***clsUsuario***.

b. Referenciar y Usar (***using***) la librería: **libCnxBD**

c. Usar (***using***) la librería: **System.Data.SqlClient**

d. Crear los atributos con tipo de dato acorde a la figura UML para dicha clase.

e. Inicializar el atributo *strCnx* con la cadena de conexión requerida para realizar la parte transaccional de la aplicación acorde a la siguiente estructura:
"Server= name_server; Database= name_bd; Integrated Security= True; TrustServerCertificate= True;"; para realizar una conexión a **SQL Server** con autenticación: **Windows Autentication**

f. Crear dos constructores
- ✓ El primer constructor sin parámetros inicializar las propiedas/atributos en cero los numéricos y en vacío los de tipo cadena, a excepción del atributo, **strCnx**.
- ✓ El segundo constructor sin parámetros e inicializar las propiedas/atributos con los respectivos valores de los parámetros de entrada.

g. Crear las propiedades con su respectivo tipo de dato y descriptor de clase acorde a la figura UML para dicha clase. Ver siguiente ejemplo:

```
public string Observacion { get; set; }
public int    Respuesta   { get; private set; }
```

clsUsuario		
-	strCnx	: string
-	strSQL	: string
-	objCnx	: clsCnxBD
-	readerLocal	: SqlDataReader
+	Codigo	: int
+	nroDoc	: int
+	tipoDoc	: int
+	Nombre	: string
+	Apellido	: string
+	Genero	: int
+	Activo	: bool
+	Observacion	: string
+	Respuesta	: int
+	strError	: string
-	validar()	: bool
+	clsUsuario( ) constructor	
+	clsUsuario(int Codigo_, int nroDocu, int tipDocu, string Nombres, string Apellidos, int idGenero, bool Activo_, string Observ)	

h. En el método **validar()**, validar que los datos de entrada: nroDoc, tipoDoc, Nombre, Apellido y Genero.

i. Crear el método **buscarMaestro(** int ***cod*** **)** con el siguiente código:

```
try
{  if ( cod <= 0 )
   {  Error = "Carné/Código no válido";
      return false;
   }
   strSQL = "EXEC USP_Usu_BuscarXCodigo '" + cod + "'; ";
   objCnx = new clsCnxBD( );
   objCnx.cadCnx = strCnx;
   objCnx.SQL = strSQL;
   if ( ! objCnx.Consultar( false ) ) //Ejecuta sincrónicamente el método Consultar
   {  Error = objCnx.Error;
      objCnx = null;
      return false;
   }
   readerLocal = objCnx.readerLleno;
   if ( ! readerLocal.HasRows )
   {  Error = "No se encontró ningún registro: " + cod;
      readerLocal.Close( );
      objCnx = null;
      return false;
   }
```


Powered by CamScanner

Institución Universitaria	GUÍA DE TRABAJO Tecnología en Desarrollo de Software	Código	FDE 048
		Versión	03
		Fecha	2009-06-09

1. IDENTIFICACIÓN

Asignatura		*Programación de Software*						Guía No.	2
Área		Básicas de la Tecnología				Nivel		4	
Código		580304006				Pensum		1	
Correquisito(s)		*580506004*				Prerrequisito(s)		NA	
Créditos	4	TPS	64	TIS	128	TPT		TIT	
TRABAJO INDEPENDIENTE						TRABAJO PRESENCIAL			
Trabajo Teórico	X	Trabajo Práctico		X		Trabajo Teórico		Trabajo Práctico	X

2. IDENTIFICACIÓN

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
Analiza, diseña e implementa aplicaciones de desarrollo de software en capas, utilizando la programación orientada a objetos con énfasis en bases de datos a partir de los requerimientos del cliente, para la solución de problemas en los sistemas de información.	✓ Preferir software para la realización de aplicaciones donde se resuelvan problema o requerimientos solicitados por el cliente de manera modular. ✓ Uso de SGBD/DBMS[1] e integración a una aplicación que requiere almacenar información de manera recurrente. ✓ Uso de la arquitectura ADO.NET para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) directamente, gestionando la base de datos de forma tradicional en una aplicación WEB.	✓ Organiza la capa de datos para la interaccionan de los formularios de la aplicación con la información existente en los SGBD, utilizando el lenguaje SQL para la manipulación de esta. ✓ Plantear diferentes medios de abstracción e interacción entre la información existente en SGBD y el usuario.

3. RECURSOS REQUERIDOS

Sala de sistemas con 40 equipos con Visual Studio 2026 y SQL Server, 1 proyector de video y marcadores varios.
Material bibliográfico

4. PROCEDIMIENTO

Programación a 3 capas
ADO.NET - (Web MVC y CRUD)

4.1 Consultar

1. ¿Qué es una programación a tres capas? Beneficios y Desventajas.
2. ¿En qué consiste la capa de base de datos?
3. ¿Qué es el patrón de arquitectura MVC en programación de software? ¿Qué hace cada una de ellas?
4. ¿Qué es CRUD en programación? ¿Qué hace cada una de ellas?
5. ¿Qué es la tecnología **ADO.NET?**
6. ¿Cuál es la arquitectura de ADO.NET?
7. ¿Cuál es el proceso para abrir y cerrar una conexión a una Base de datos SQL 2005/2008 o superior desde C#?
8. ¿Cuáles son los objetos básicos para manipular la información en una base de datos de ADO.NET?
9. ¿Cuáles son las librerías de .NET e instrucciones de C# necesarias para realizar una conexión a bases de datos SQL 2005/2008 o superior?
10. ¿Como documentar Propiedades y Métodos en una clase en un proyecto .NET
11. ¿Cuál es la función de los objetos: **¿SqlDataReader, SqlDataAdapter, SqlDataSet y DataTable** en la tecnología ADO.NET?
12. ¿En la tecnología .NET? Cuáles son las propiedades y los métodos necesarios para utilizar un: **SqlConnection, SqlCommand, sqlDataReader y DataSet?**
13. ¿Cuales son los tres tipos de ejecución de un **SqlCommand? , cual es la función de cada uno.**
14. ¿Cuáles son las similitudes y las diferencias entre un **SqlDataReader** y un **SqlDataSet?**
15. ¿En la tecnología .NET, Cómo se llena un: **sqlDataReader** y ***DataSet?.***
16. ¿Cómo crear un usuario en Microsoft SQL Server e instanciarlo a una Base de datos determinada?

4.2 Problema para solucionar

SuperMercado SAS requiere de un formulario para registrar a los Usuarios. Debe contener el código de usuario, el número y el tipo de documento, el/los nombre, el/los apellidos, el género (10, Masculino o 20, Femenino), activo y observaciones. El formulario debe contener la forma de poder realizar las opciones de buscar, ingresar, modificar, grabar y cancelar. Los tipos de documentos con sus respectivos id: (1, Cédula Ciudadanía), (3, Cédula Extranjería), (6, Pasaporte), (7, NIT), (8, NUIP), (9, Permiso de Protección Temporal).

[1] Sistema de Gestión de Bases de Datos (en inglés: DataBase Management System)

Powered by CamScanner

Actividad # 1. Bases de Datos:

1.1. Crear una carpeta con el nombre: ***Practica2_SuperMkdo***

1.2. Crear una subcarpeta con nombre: ***BasedeDatos***

1.3. Crear un script de base de datos en el DBMS SQL Server y guárdalo en la subcarpeta creada en el numeral anterior, con nombre: ***bdSuperMkdo***; dicho script debe contener los siguientes ítems:

i. Crear y usar la base de datos: **bdSuperMkdo** (colocar la instrucción: **go** al final de cada estructura, instrucción o grupo de instrucciones)

```
Create Database bdSuperMkdo
go
USE bdSuperMkdo
go
```

ii. Crear la tabla: **tblUsuario**

```
Create Table tblUsuario
( codigo int Primary Key NOT NULL,
  nroDoc int NOT NULL,
  idTipDoc int NOT NULL,
  nombre varchar(50) NOT NULL,
  apellido varchar(50) NOT NULL,
  genero int NOT NULL,
  activo bit default 1 NOT NULL,
  observac varchar(500) NULL
)
go
```

tblUsuario
- codigo
- nroDoc
- idTipDoc
- nombre
- apellido
- genero
- activo
- observac

iii. Ingresar datos a la tabla:

```
--============ tblUsuario ============
INSERT INTO tblUsuario VALUES
 (30001, 1020001000, 7, 'Claudia',   'Aristizábal Rúa',  20, 1, 'Ppto Particip'),
 (30002, 1020001001, 8, 'Mario A.',  'Martínez Rios',    10, 1, 'Fondo EPM'),
 (30003, 1020001002, 1, 'Natalia',   'Castrillón Caguán', 20, 1, null),
 (30004, 1020001003, 1, 'Rubén D.',  'Soto Rios',        10, 1, 'Pruebas de todo'),
 (30005, 1020001004, 3, 'Scott',     'Rangún Suteu',     10, 1, 'Reserva'),
 (30006, 1020001005, 9, 'Jazmín A.', 'Arango Marín',     20, 1, 'Ppto Particip.'),
 (30007, 1020001006, 6, 'Cristina M.','Soto Diaz',       20, 1,'Suspende Matric')
go
```

iv. Crear Consultas almacenadas (Stored Procedures) :

```
Create Procedure USP_Usu_BuscarXCodigo
@intCodigo int
AS
 BEGIN
  SELECT nroDoc, idTipDoc, nombre, apellido, genero, Activo, isnull( observac,'' ) observac
    FROM tblUsuario
   WHERE codigo = @intCodigo
  -- EXEC USP_Usu_BuscarXCodigo 30001;
 END
go
```

```
Create Procedure USP_Usu_Grabar
@intNroDoc   int,
@intTipDoc   int,
@strNombre   VARCHAR(50),
@strApelli   VARCHAR(50),
@intGenero   int,
@bitActivo   BIT,
@strObserv   VARCHAR(500)
As
 BEGIN
  IF EXISTS( SELECT * FROM tblUsuario
             WHERE nroDoc = @intNroDoc )
   BEGIN
    SELECT -1 AS Rpta
    Return
   END
  ELSE
   BEGIN
    BEGIN TRANSACTION tx
     Declare @intNewCod int
     Set @intNewCod = ( select max( codigo ) + 1
                        from tblUsuario )
     INSERT INTO tblUsuario VALUES ( @intNewCod,
          @intNroDoc, @intTipDoc, @strNombre, @strApelli,
```

```
Create Procedure USP_Usu_Modificar
@intCodigo   int,
@intNroDoc   int,
@intTipDoc   int,
@strNombre   VARCHAR(50),
@strApelli   VARCHAR(50),
@intGenero   int,
@bitActivo   BIT,
@strObserv   VARCHAR(500)
As
 BEGIN
  IF EXISTS( SELECT * FROM tblUsuario
             WHERE nroDoc = @intNroDoc and codigo <> @intCodigo )
   BEGIN
    SELECT -1 AS Rpta
    Return
   END
  ELSE
   BEGIN
    BEGIN TRANSACTION tx
     UPDATE tblUsuario SET
          nroDoc = @intNroDoc, idTipDoc = @intTipDoc,
          nombre = @strNombre, apellido = @strApelli,
          genero = @intGenero, Activo = @bitActivo,
          Observac = @strObserv
```


Powered by CamScanner

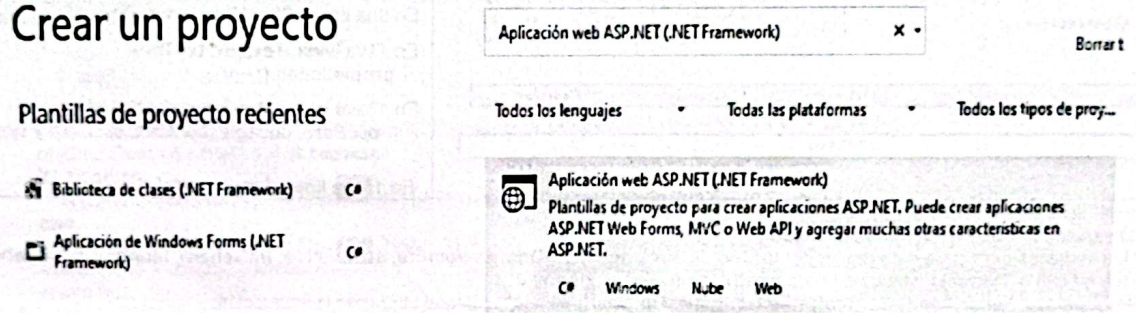
```
        @intGenero, @bitActivo, @strObserv );
    IF ( @@ERROR > 0 )
     BEGIN
       ROLLBACK TRANSACTION tx
       SELECT 0 AS Rpta
       Return
     END
     COMMIT TRANSACTION tx
     SELECT @intNewCod AS Rpta
     Return
   END
  /* EXEC USP_Usu_Grabar '70500600', 1, 'Andrés', 'Duque', 10, 1, null;
    EXEC USP_Usu_BuscarXcodigo '30008';      */
 END
go
```

```
      WHERE codigo = @intCodigo
     IF ( @@ERROR > 0 )
      Begin
        ROLLBACK TRANSACTION tx
        SELECT 0 AS Rpta
        Return
      End
      COMMIT TRANSACTION tx
      SELECT @intCodigo  AS Rpta
      Return
   END
  /* EXEC USP_Usu_BuscarXcodigo '30008';
 EXEC USP_Usu_Modificar '30008', '70500600', 6, 'Antonia', 'Duque', 20, 1, 'test';
    EXEC USP_Usu_BuscarXcodigo '30008';      */
 END
go
```

Actividad 2. Crear Proyecto WEB en C# con formulario

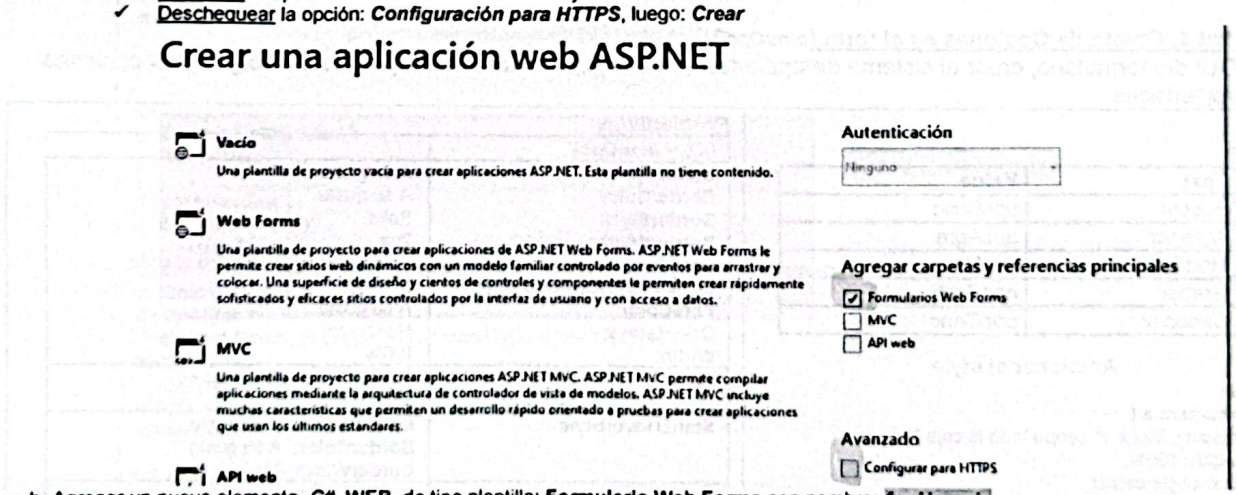
2.1. Aplicación para entorno WEB para realizar la captura y muestra de datos, utilizando las librerías de clases implementadas anteriormente

a. En Microsoft Visual Studio 2026, utilizar la opción: Crear un Proyecto, buscando o utilizando la plantilla de proyecto: *Aplicación web ASP.NET (.NET Framework)* en C#, con nombre: ***webSuperMkdo***, en la subcarpeta creada en la actividad 1(1.1). ***Crear.***

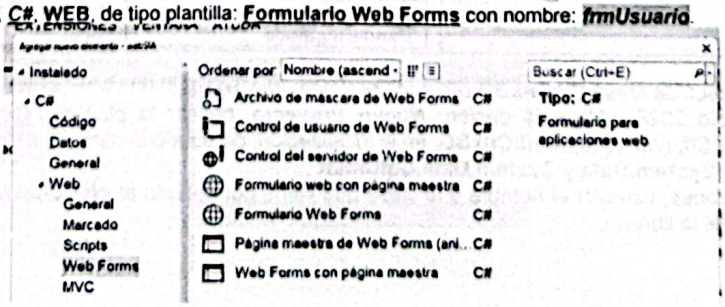


Te lleva a la ventana de: *Crear una aplicación web ASP.NET*, en ella seleccionar: ***Empty*** (Vacío) y

- ✓ Chequear la opción: ***Formularios Web Forms*** y
- ✓ Deschequear la opción: ***Configuración para HTTPS***, luego: ***Crear***



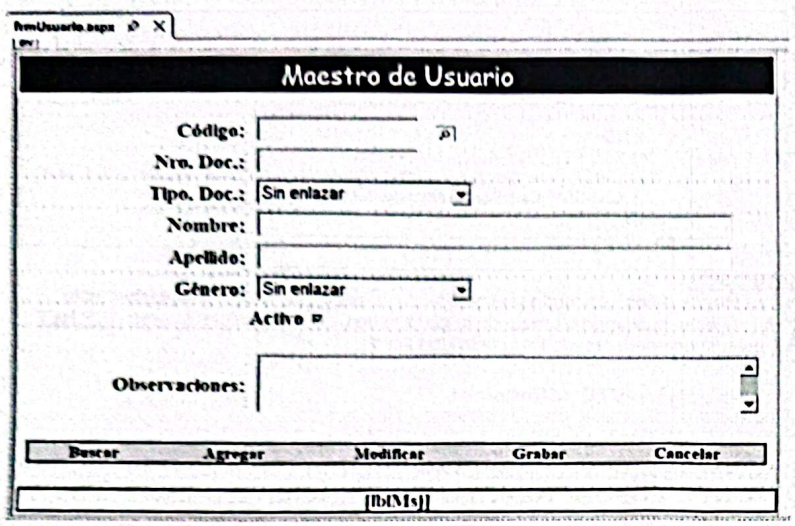
b. Agregar un nuevo elemento, C#, WEB, de tipo plantilla: **Formulario Web Forms** con nombre: ***frmUsuario***.



Powered by CamScanner

c. En el proyecto: Agregar -> Nueva carpeta, con nombre: ***Imagenes***; en ella adicionar la imagen entregada por el docente. (Pararse en la carpeta creada, Agregar -> Elemento existente, buscar en donde tiene la imagen y adicionarla).

d. En ***frmUsuario*** diseñar los objetos de entrada y salida propuestos en la siguiente imagen, utilizando una tabla de 15*2, con ancho del 90%, centrada.



Form: ***frmUsuario***
(*Titulo/title*): Maestro de Usuario

En 1ra línea: Unirla en un td, (Text): ***Maestro de Usuario***

Ubicar los siguientes objetos en 2do td de cada línea

En 3ra línea, *Textbox:* **txtCodI**, ReadOnly: true, luego un, *ImageButton:* **IbtnBusc**, en propiedad: *ImageUrl* seleccionar la imagen de la carpeta: Imagenes.

En 4ta línea, *Textbox*: **txtNDoc**

En 5ta línea, *DropDownList*: **ddlTDoc**

En 6ta línea, *Textbox*: **txtNomb**

En 7ma línea, *Textbox*: **txtApel**

En 8va línea, *DropDownList*: **ddlGene**

En 9na línea, *CheckBox*: **chkActI** Text: Activo y TextAlign: Left)

En 11va línea, *Textbox*: **txtObse**
propiedades:(TextMode: MultiLine y Rows: 3)

En 13ava línea, Menú: **mnuOpcI**, *Values*:
opcBusc, opcAgre, opcModi, opcGrab y opcCanc. (Ver Actividad 3)

En 15ava línea, Label: **lblMsj, (center, red)**

e. Establecer esta página como **página de inicio**

f. Crear la variable global: ***intOpcion*** de tipo **static**

g. Crear las variables para cada dato requerido: **intCod, intNroDoc, intTipDoc, strNombre, strApellido, intGenero, blnAct y strObservac.**

h. Crear los métodos: **Mensaje**(string txt) para mostrar mensajes en: *lblMsj*

i. Crear los métodos: **llenarComboGenero()** y **llenarComboTipoDoc()** de forma manual utilizando la instrucción:
this.ddlGene.Items.Add(new ListItem("Seleccionar...", "0")); con los datos en el enunciado respectivamente.

j. En el método Page_Load, en la negación del postBack, invocar los métodos: **llenarComboProg( ) y llenarComboTipoDoc()**.

Actividad 3. Objeto de Opciones en el form (*mnuOpci*)

En la GUI del formulario, crear el sistema de opciones con el objeto: Menú, ***mnuOpci;*** con las siguientes opciones y propiedades:

Items

Opciones	Text	Value
	Buscar	opcBusc
	Agregar	opcAgre
	Modificar	opcModi
	Grabar	opcGrab
	Cancelar	opcCanc

Adicionar el style

```
<style>
  .menu-item a {
      display: block; /* ocupe toda la caja */
      width: 100%;
      text-align: center;
  }
</style>
```

Propiedades:

(ID) = *mnuOpci*	
BackColor:	A su gusto
BorderColor:	A su gusto
BorderStyle:	Solid
BorderWidth:	2px
CssClass:	***menu-item*** //Creado en el style
DynamicHorizontalOffset:	2
ForeColor:	A su gusto
Orientation:	Horizontal
Width:	100%
Font:	**Bold** = True **Size** = Medium
StaticHoverStyle	**BackColor:** A su gusto **BorderColor:** A su gusto **BorderWidth:** 2px
StaticMenuItemStyle	**HorizontalPadding:** 20px
RenderingMode	Table

Actividad 4. Crear la biblioteca de clase (***libCnxBD.dll***) con las Clase: ***clsCnxBD.***

a En Microsoft Visual Studio 2026, utilizar la opción: ***Nuevo Proyecto***, buscar la plantilla: ***Biblioteca de clases (.NET Framework)***, con ***Visual C#***, con nombre: ***libCnxBD***, en la localización: carpeta creada en la actividad 1(1.1). Aceptar

b. Usar (***using***) las librerías: **System.Data y System.Data.SqlClient**

c. En el explorador de soluciones, cambiar el nombre a la clase que viene por defecto a: ***clsConexion***, grabar para que quede así guardado el archivo de la librería.

Powered by CamScanner

d. Crear constructor donde inicialice ***blnHayCnx*** en falso, y crear los objetos: ***objCnx*** desde SqlConnection, ***objCmd*** desde SqlCommand y ***objParam*** desde SqlParameter.

e. Crear el método privado: ***abrirCnx***

```
if ( string.IsNullOrEmpty( cadCnx ) )
   return false;
objCnx.ConnectionString = cadCnx;
try
{  objCnx.Open( );
   blnHayCnx = true;
   return true;
}
catch ( Exception ex )
{  Error = "Error, reintentar";
   blnHayCnx = false;
   return false;
}
```

f. Crear el método público: ***cerrarCnx***

```
try
{  objCnx.Close( );
   objCnx = null;
   blnHayCnx = false;
}
catch
{  Error = "Problemas en liberación de la conexión." }
```

clsCnxBD		
-	blnHayCnx	: bool
-	objCnx	: SqlConnection
-	objCmd	: SqlCommand
-	objParam	: SqlParameter
+	SQL	: string ↓
+	cadCnx	: string ↓
+	readerLleno	: SqlDataReader ↑
+	vrRpta	: object ↑
+	Error	: string ↑
-	abrirCnx( )	: bool
+	Consultar(bool tipoEjec)	: bool
+	vrUnico(bool tipoEjec)	: bool
+	execCmd(bool tipoEjec)	: bool
+	addParam(string nombreParam, object vrParam)	: bool
+	clsCnxBD( ) // constructor	

g. Crear el método público: ***Consultar*** con el parámetro, ***tipoEjec*** de tipo lógico

```
try
{  if ( string.IsNullOrEmpty( SQL ) || string.IsNullOrEmpty( cadCnx ) )
   {  Error = "Faltan datos";
      return false;
   }
   if ( ! blnHayCnx )
      if ( ! abrirCnx( ) )
         return false;
   objCmd.Connection = objCnx;  //Preparar el Comando para ejecutar la transacción SQL en la BD
   objCmd.CommandText = SQL;
   if ( tipoEjec)
      objCmd.CommandType = CommandType.StoredProcedure;  //Asincrónico
   else
      objCmd.CommandType = CommandType.Text;              //Sincronico
   readerLleno = objCmd.ExecuteReader( );  //Realizar la transacción en la BD
   return true;
}
catch ( Exception ex )
{  Error = ex.Message;
   return false;
}
```

h. Crear el método público: ***vrUnico*** con el parámetro, ***tipoEjec*** de tipo lógico

```
try
{  if ( string.IsNullOrEmpty( SQL ) || string.IsNullOrEmpty( cadCnx ) )
   {  Error = "Faltan datos";
      return false;
   }
   if ( ! blnHayCnx )
      if ( ! abrirCnx( ) )
         return false;
   objCmd.Connection = objCnx;  //Preparar el Comando para ejecutar la transacción SQL en la BD
   objCmd.CommandText = SQL;
   if ( tipoEjec )
      objCmd.CommandType = CommandType.StoredProcedure;
   else
      objCmd.CommandType = CommandType.Text;
   vrRpta = objCmd.ExecuteScalar();  //Realizar la transacción en la BD
   return true;
}
catch ( Exception ex )
{  Error = ex.Message;
   return false;
}
```

i. Crear el método público: ***execCmd*** con el parámetro, ***tipoEjec*** de tipo lógico

```
try
{  if ( string.IsNullOrEmpty( SQL ) || string.IsNullOrEmpty( cadCnx ) )
   {  Error = "Faltan datos";
      return false;
   }
   if ( ! blnHayCnx )
      if ( ! abrirCnx( ) )
         return false;
```


Powered by CamScanner

```
objCmd.Connection = objCnx;    //Preparar el Comando para ejecutar la transacción SQL en la BD
objCmd.CommandText = SQL;
if ( tipoEjec )
   objCmd.CommandType = CommandType.StoredProcedure;
else
   objCmd.CommandType = CommandType.Text;
objCmd.ExecuteNonQuery( );  //Realizar la transacción en la BD
return true;
}
catch ( Exception ex )
{  Error = ex.Message;
   return false;
}  string nombreParam, object
```

j. Crear el método pùblico: ***addParam*** con los parámetros, ***nombreParam*** de tipo cadena y ***vrParam*** de tipo object

```
try {
   objCmd.Parameters.AddWithValue( nombreParam, vrParam ); //Adicionar el parámetro al command
   objParam = new SqlParameter( );    //Crear un nuevo parámetro para el siguiente parámetro
   return true;
}
catch
{  Error = "Error en ingreso de dato.";
   return false;
}
```

Actividad 5. En el Proyecto *web ASP.NET*: ***webSuperMkdo***, crear la clase local: ***clsUsuario***

a En el explorador de soluciones, en el proyecto, agregar una *nueva carpeta* con nombre: ***Clases***. En dicha carpeta Agregar una nueva clase local con nombre: ***clsUsuario***.

b. Referenciar y Usar (***using***) la librería: **libCnxBD**

c. Usar (***using***) la librería: **System.Data.SqlClient**

d. Crear los atributos con tipo de dato acorde a la figura UML para dicha clase.

e. Inicializar el atributo *strCnx* con la cadena de conexión requerida para realizar la parte transaccional de la aplicación acorde a la siguiente estructura:
"**Server= name_server; Database= name_bd; Integrated Security= True; TrustServerCertificate= True;**"; para realizar una conexión a **SQL Server** con autenticación: **Windows Autentication**

f. Crear dos constructores
- ✓ El primer constructor sin parámetros inicializar las propiedas/atributos en cero los numéricos y en vacío los de tipo cadena, a excepción del atributo, **strCnx**.
- ✓ El segundo constructor sin parámetros e inicializar las propiedas/atributos con los respectivos valores de los parámetros de entrada.

g. Crear las propiedades con su respectivo tipo de dato y descriptor de clase acorde a la figura UML para dicha clase. Ver siguiente ejemplo:

```
public string Observacion { get; set; }
public int    Respuesta   { get; private set; }
```

clsUsuario		
-	strCnx : string	
-	strSQL : string	
-	objCnx : clsCnxBD	
-	readerLocal : SqlDataReader	
+	Codigo : int	↑↓
+	nroDoc : int	↑↓
+	tipoDoc : int	↑↓
+	Nombre : string	↑↓
+	Apellido : string	↑↓
+	Genero : int	↑↓
+	Activo : bool	↑↓
+	Observacion : string	↑↓
+	Respuesta : int	↑
+	strError : string	↑
-	validar() : bool	
+	clsUsuario( ) constructor	
+	clsUsuario(int Codigo_, int nroDocu, int tipDocu, string Nombres, string Apellidos, int idGenero, bool Activo_, string Observ)	

h. En el método **validar()**, validar que los datos de entrada: nroDoc, tipoDoc, Nombre, Apellido y Genero.

i. Crear el método **buscarMaestro**(int ***cod***) con el siguiente código:

```
try
{  if ( cod <= 0 )
   {  Error = "Carné/Código no válido";
      return false;
   }
   strSQL = "EXEC USP_Usu_BuscarXCodigo '" + cod + "';";
   objCnx = new clsCnxBD( );
   objCnx.cadCnx = strCnx;
   objCnx.SQL = strSQL;
   if ( ! objCnx.Consultar( false ) ) //Ejecuta sincrónicamente el método Consultar
   {  Error = objCnx.Error;
      objCnx = null;
      return false;
   }
   readerLocal = objCnx.readerLleno;
   if ( ! readerLocal.HasRows )
   {  Error = "No se encontró ningún registro: " + cod;
      readerLocal.Close( );
      objCnx = null;
      return false;
   }
```


Powered by CamScanner

```
    readerLocal.Read( );
    nroDoc = readerLocal.GetInt32( 0 );      tipoDoc = readerLocal.GetInt32( 1 );      Nombre = readerLocal.GetString( 2 );
    Apellido = readerLocal.GetString( 3 );   Genero = readerLocal.GetInt32( 4 );       Activo = readerLocal.GetBoolean( 5 );
    Observacion = readerLocal.GetString( 6 );
    readerLocal.Close( );
    objCnx = null;
    return true;
  }
  catch ( Exception ex )
  {  Error = ex.Message;
     return false;
  }
```

j. En el método **Grabar(int Accion)**, adicionar el siguiente código:

```
try
{  objCnx = new clsCnxBD( );
   objCnx.cadCnx = strCnx;
   objCnx.SQL = strSQL;
   if ( Accion == 2 ) {   // Modificar          //Enviar parámetros requeridos por el USP
      if ( ! objCnx.addParam( "@intCodigo",  Codigo ) ) {
         Error = objCnx.Error;
         objCnx = null;
         return false;
      }
   }
   else if ( Accion != 1 ) {
      Error = "Acción no válida";
      return false;
   }
```

Enviar el resto de los parámetros cada uno con el nombre del parámetro en el stored procedure (@intNroDoc, @intTipDoc, @strNombre, @strApelli, @intGenero, @bitActivo y @strObserv), y el valor de la propiedad/atributo. **Ver numeral anterior como ejemplo y siguiente como plantilla**.

```
   if ( ! objCnx.addParam( "parametro", valorEnviar )
   {  Error = objCnx.Error;
      objCnx = null;
      return false;
   }

   if ( ! objCnx.vrUnico( true ) )   // Ejecutar stored procedure asincrónicamente  --> true
   {  Error = objCnx.Error;
      objCnx = null;
      return false;
   }
   Respuesta = Convert.ToInt32( objCnx.vrRpta );
   objCnx = null;
   return true;
}
catch (Exception ex)
{  Error = ex.Message;
   return false;
}
```

k. En los métodos **grabarMaestro( ) y modificarMaestro( )** y , adicionar el siguiente código:

grabarMaestro( )	modificarMaestro( )

if (! validar( ))
  return false;
strSQL = "USP_Usu_Grabar";
return Grabar(1);
	if (! validar( ))
	  return false;
	if (Codigo <= 0)
	{ Error = "Nro. de Código no válido";
	  return false;
	}
	strSQL = "USP_Usu_Modificar";
	return Grabar(2);

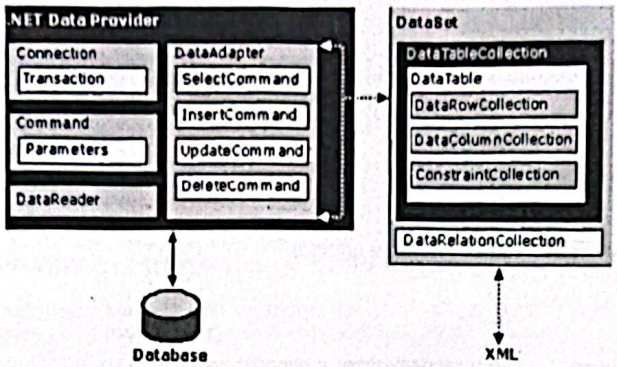
Actividad 6. En la página web

a. Crear el método: **Limpiar( )** para borrar y posicionar la info en cada objeto del formulario.
b. Crear el método: **Buscar(int Codigo)** para buscar acorde al código y mostrar lo encontrado en el formulario.
c. Crear el el método: **Grabar()** para capturar, crear objeto a partir de la clase: ***clsUsuario*** y acorde a la acción a realizar ejecute la transacción necesaria a grabar y mostrar el mensaje de la acción realizada.
d. Codificar los eventos de los objetos: **mnuOpcl_MenuItemClick y ibtnBusc_Click.**

Powered by CamScanner

5. Tema de estudio

Tecnología ADO.NET



Entre los objetos de ADO.NET tenemos los siguientes:

- Objetos **Connection**. Para conectar con una base de datos y administrar las transacciones en una base de datos.
- Objetos **Command**. Para emitir comandos SQL a una base de datos.
- Objetos **DataReader**. Proporcionan una forma de leer una secuencia de registros de datos sólo hacia delante desde un origen de datos SQL Server.
- Objetos **DataSet**. Para almacenar datos sin formato, datos XML y datos relacionales, así como para configurar el acceso remoto y programar sobre datos de este tipo.
- Objetos **DataAdapter**. Para insertar datos en un objeto **DataSet** y reconciliar datos de la base de datos.

Elegir un DataReader o un DataSet

A la hora de decidir si su aplicación debe utilizar un DataReader o un DataSet, DataTables o DataViews, debe tener en cuenta el tipo de funcionalidad que su aplicación requiere. Use un DataSet para hacer lo siguiente:

- Almacene datos en la memoria caché de la aplicación para poder manipularlos. Si solamente necesita leer los resultados de una consulta, el *DataReader* es la mejor elección.
- Utilizar datos de forma remota entre un nivel y otro o desde un servicio Web XML.
- Interactuar con datos dinámicamente, por ejemplo, para enlazar con un control de Windows Forms o para combinar y relacionar datos procedentes de varios orígenes.
- Realizar procesamientos exhaustivos de datos sin necesidad de tener una conexión abierta con el origen de datos, lo que libera la conexión para que la utilicen otros clientes.

Si no necesita la funcionalidad proporcionada por el *DataSet*, puede mejorar el rendimiento de su aplicación si utiliza el *DataReader* para devolver sus datos de solo avance y de solo lectura. Aunque *DataAdapter* utiliza *DataReader* para rellenar el contenido de un *DataSet*, al utilizar el *DataReader* puede mejorar el rendimiento porque no usará la memoria que utilizaría el *DataSet*, además de evitar el procesamiento necesario para crear y rellenar el contenido de *DataSet*.

ADO.NET aprovecha la eficacia de XML para proporcionar acceso a datos sin conexión. ADO.NET fue diseñado teniendo en cuenta las clases de XML incluidas en .NET Framework; ambos son componentes de una única arquitectura.

ADO.NET y las clases de XML incluidas en .NET Framework convergen en el objeto *DataSet*. *DataSet* se puede rellenar con datos procedentes de un origen XML, ya sea éste un archivo o una secuencia XML. *DataSet* se puede escribir como XML conforme al consorcio World Wide Web (W3C), que incluye su esquema como esquema lenguaje de definición de esquemas XML, independientemente del origen de los datos incluidos en *DataSet*. Puesto que el formato nativo de serialización del *DataSet* es XML, es un medio excelente para mover datos de un nivel a otro, por lo que *DataSet* es idóneo para usar datos y contextos de esquemas de comunicación remota desde y hacia un servicio web XML.

Tomado de: https://msdn.microsoft.com/es-es/library/27y4ybxw(v=vs.110).aspx

Diferentes cadenas de Conexión

En el siguiente enlace, observar la sintaxis de algunas de las cadenas de conexión empleadas para conectar nuestras aplicaciones a diferentes motores de bases de datos, entre ellas: *Oracle*, *SQL Server*, *Acces*, *Excel*, *DB2*, *MySQL*, etc. Es de un apoyo enorme al momento de realizar instancias desde nuestras aplicaciones a los diferentes motores de bases de datos.

http://www.connectionstrings.com/

Cadena de conexión con autenticación de Windows

Para conectar a una base de datos de SQL Server con autenticación de Windows, la cadena de conexión será:
Nota: Observar que cada parámetro con su valor, están separados por punto y coma.

Server = ServidorSQL; **Database** = BaseDatos; **Integrated Security** = **SSPI**;

Nota: Observar que cada parámetro con su valor, están separados por punto y coma.

Cadena de conexión con autenticación de SQL Server

Para conectar a una base de datos de SQL Server usando autenticación del propio SQL Server, la cadena de conexión será:

Data Source = ServidorSQL; **Initial catalog** = BaseDatos; **user id** = Usuario; **password** = Contraseña;

Powered by CamScanner

MVC (Modelo-Vista-Controlador)

Es un patrón de arquitectura en el diseño de software comúnmente utilizado para implementar interfaces de usuario, datos y lógica de control. Enfatiza una separación entre la lógica de negocios y su visualización. Esta "separación de preocupaciones" proporciona una mejor división del trabajo, una mejora de mantenimiento, escalabilidad y desarrollo en equipo. Algunos otros patrones de diseño se basan en MVC, como MVVM (Modelo-Vista-modelo de vista), MVP (Modelo-Vista-Presentador) y MVW (Modelo-Vista-Whatever).

Los tres componentes principales de MVC:

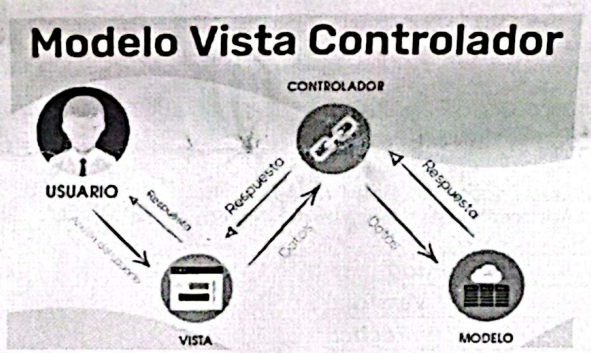
- ✓ Modelo (Model - datos/lógica): Gestiona los datos y la lógica de negocio. Se encarga de la base de datos, consultas, validaciones y manipulación de la información.
- ✓ Vista (View - interfaz gráfica): Es la representación visual de los datos, lo que el usuario ve y con lo que interactúa, como la página HTML o la interfaz gráfica
- ✓ Controlador (Controller - intermediario): Actúa como intermediario. Recibe las acciones del usuario (peticiones HTTP), interactúa con el Modelo para obtener los datos necesarios y selecciona la Vista para mostrar la respuesta.

¿Cómo funciona?

- ✓ El usuario interactúa con la interfaz (Vista), por ejemplo, haciendo clic en un enlace.
- ✓ El Controlador recibe la petición.
- ✓ El Controlador solicita al Modelo que actualice o busque información.
- ✓ El Modelo procesa los datos y los devuelve al Controlador.
- ✓ El Controlador envía los datos a la Vista para su representación final.

Ventajas de usar MVC:

- ✓ Separación de responsabilidades: Facilita el mantenimiento al tener código organizado.
- ✓ Reutilización: Permite usar diferentes vistas para los mismos datos.
- ✓ Desarrollo en paralelo: Diferentes desarrolladores pueden trabajar en el modelo, la vista y el controlador simultáneamente.
- ✓ Frameworks populares: Se utiliza en herramientas como Laravel, Django, Ruby on Rails y ASP.NET.



CRUD

Es un acrónimo de **Create (Crear), Read (Leer), Update (Actualizar) y Delete (Eliminar)**, las cuatro operaciones fundamentales que se realizan sobre los datos en un sistema de almacenamiento como una base de datos o una aplicación, permitiendo gestionar el ciclo de vida completo de la información. Estas operaciones corresponden a las sentencias SQL INSERT, SELECT, UPDATE, y DELETE, respectivamente, y son la base para la manipulación de datos en cualquier software.

Operaciones CRUD

- ✓ **Create** (Crear): Insertar nuevos registros o datos en la base de datos, como añadir un nuevo usuario o producto.
- ✓ **Read** (Leer):
- ✓ Recuperar o consultar datos existentes, como ver el perfil de un usuario o la lista de productos.
- ✓ **Update** (Actualizar): Modificar datos que ya existen en la base de datos, como cambiar la dirección de un cliente o editar el título de un post.
- ✓ **Delete** (Eliminar): Borrar registros de la base de datos, como dar de baja a un usuario o eliminar un comentario.

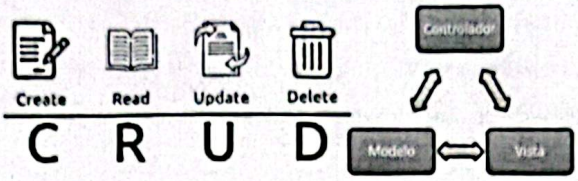
Ejemplos prácticos

- ✓ En una lista de tareas (To-Do List): Añadir una nueva tarea (Create), ver las tareas pendientes (Read), marcar una tarea como completada (Update), borrar una tarea (Delete).
- ✓ En una red social: Publicar una foto (Create), ver el feed de noticias (Read), editar tu foto de perfil (Update), eliminar una publicación (Delete).

Powered by CamScanner

Importancia

- ✓ Fundamento: Es el pilar para el manejo de datos en casi cualquier aplicación, desde blogs hasta sistemas empresariales complejos.
- ✓ Claridad: Estructura el código de manera ordenada y facilita la comprensión del flujo de datos.
- ✓ Extensibilidad: Permite la creación de aplicaciones robustas y escalables, siendo un concepto clave en el desarrollo de APIs (Interfaces de Programación de Aplicaciones) y microservicios.



6. PARÁMETROS PARA ELABORACIÓN DEL INFORME

NA.

7. BIBLIOGRAFÍA

- ✓ Deitel, Harvey M.; Deitel, Paul J. Cómo programar en C#. - 2. Ed., México: Pearson Educación, 2007, 1166 p. 1CD ROM, ISBN: 9702610567, (ITM) 005.133 D325
- ✓ Charte Ojeda, Francisco; Serrano Pérez, Jorge. Programación con Visual Studio .NET, Madrid: Anaya Multimedia, 2002, 654 p. + CD-ROM, ISBN: 8441513767, (ITM) 005.133 C486
- ✓ Kingsley-Hughes, Adrian; Kingsley-Hughes, Kathie. C# 2005: Aprendizaje y referencia, Madrid: Anaya Multimedia, 2007, 448 p, ISBN: 9788441522381, (ITM) 005.133 K55
- ✓ Bradley, Julia Case; Millspaugh, Anita C. Programming in C# .NET, New York: McGraw-Hill, 2003, 556p. + CD-ROM, ISBN: 0071215646, (ITM) 005.133 B811
- ✓ Wright, Charles. Superutilidades para C#, Madrid: McGraw-Hill, 2003, 639p, ISBN: 9789701047834, (ITM) 005.133 W948
- ✓ Ramírez, Felipe. Introducción a la programación: algoritmos y su implementación en VB.NET, C#, Java y C++. - 2. Ed, México: Alfaomega, 2007, 488 p, ISBN: 9789701512807, (ITM) 005.1 R173
- ✓ Charte Ojeda, Francisco. Visual C# 2005, Madrid: Anaya Multimedia, 2007, 351 p, ISBN: 8441520836, (ITM) 005.133 C486
- ✓ Foxall, James. Visual C# 2005, Madrid: Anaya Multimedia, 2007, 559 p, ISBN: 9788441521216, (ITM) 005.133 F792
- ✓ Ceballos Sierra, Francisco Javier. El lenguaje de programación C#, México: Alfaomega, 2007. 302 p. CD-ROM, ISBN: 9701506017, (ITM) 005.133 C387.
- ✓ Wille, Christoph. C#, Madrid: Prentice-Hall, 2001, 187 p, ISBN: 8420531138, (ITM) 005.133 W698
- ✓ Ceballos Sierra, Francisco Javier. Enciclopedia de Microsoft Visual C#, México: McGraw-Hill, 2006, 936 p. + CD-ROM, ISBN: 9701512170, (ITM) 005.133 C387e

Elaborado por:	*Delio Augusto Aristizábal Martínez*
Versión:	*2*
Fecha:	*Agosto 2026*
Aprobado por:	

8. Taller

1. Crear un formulario maestro para los productos de una cafetería que contiene el código, La descripción, el valor unitario, la cantidad de existencia, porcentaje de descuento, porcentaje de IVA, valor del costo y activo.
2. Crear un formulario para la venta de un único producto que contiene el número de la factura (automático), fecha de elaboración, número del documento y nombres del cliente, lista de los productos, cantidad, valor unitario, porcentaje de descuento, porcentaje de IVA.
3. Crear un formulario para realizar transacciones a la siguiente tabla que no se encuentra normalizada.

Ordenes

Id orden	Fecha	Id cliente	Nom cliente	Estado	Num art	nom art	cant	Precio
2301	23/02/11	101	Martin	Caracas	3786	Red	3	35,00
2301	23/02/11	101	Martin	Caracas	4011	Raqueta	6	65,00
2301	23/02/11	101	Martin	Caracas	9132	Paq-3	8	4,75
2302	25/02/11	107	Herman	Coro	5794	Paq-6	4	5,00
2303	27/02/11	110	Pedro	Maracay	4011	Raqueta	2	65,00
2303	27/02/11	110	Pedro	Maracay	3141	Funda	2	10,00

Powered by CamScanner